Aider 的工作原理

Lab01 · 设计你的结对程序员 · 2026 年 9 月 14–15 日

layout: default

Lab01

日程

1. Aider 深度剖析
2. Hackathon 1
3. 下一个作业
4. 办公时间

layout: section

框架

你和模型之间的程序

layout: default

你已经知道的框架

你已经使用过四个框架 {.assert}

	放在模型前面	对回复做什么
ChatGPT	你的消息，以及它记录的笔记	展示给你
Cursor	你指向的文件，以及一个索引	你接受的 diff
Claude Code	它自己读取的文件，以及你保存的笔记	如果允许则编辑或运行命令
Aider	你添加的文件，以及一张地图	写入，然后提交

一种模型在四个工具后面。框架是不同的部分。

layout: default

什么是框架

框架是模型周围的一切 {.assert}

<ph-stack-bold class="text-3xl text-blue-600 mb-3" />
组装
挑选模型将看到的每一个词。
<ph-chat-text-bold class="text-3xl text-amber-600 mb-3" />
回答
模型写一条回复。文本，仅此而已。
<ph-pencil-simple-bold class="text-3xl text-blue-600 mb-3" />
应用
决定回复可以改变什么，然后进行改变。

layout: default

什么是框架

模型永远不会接触什么

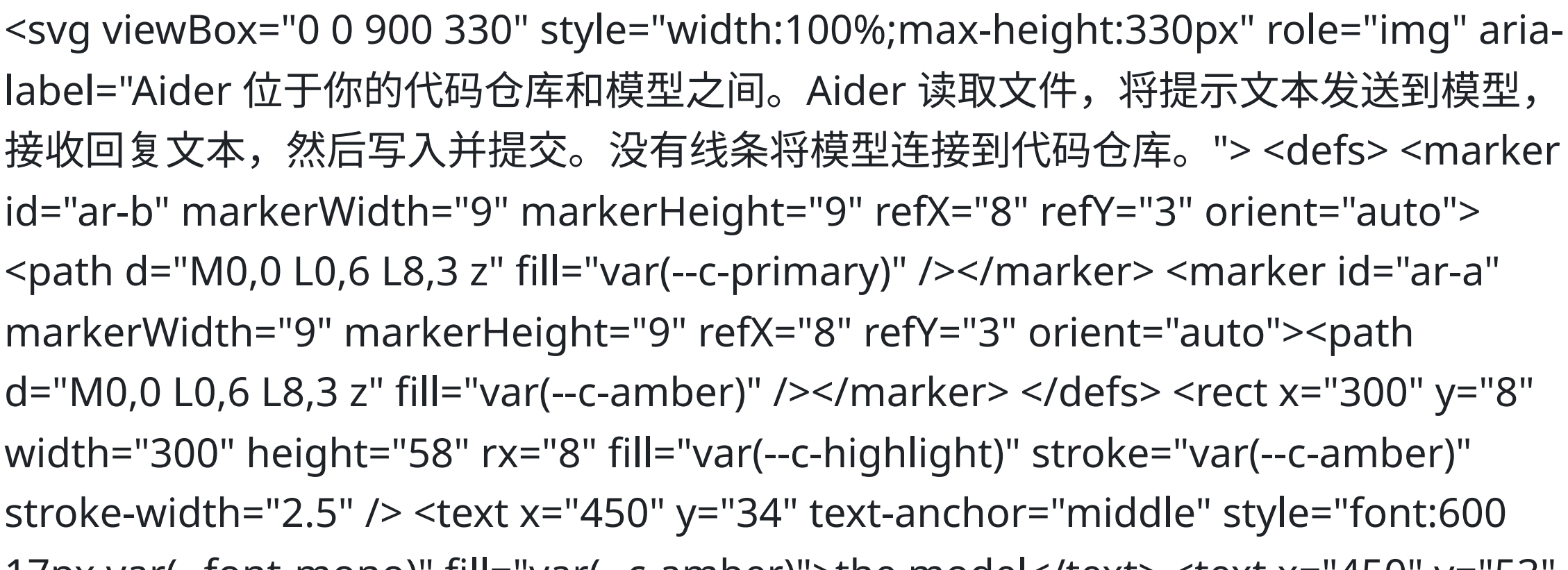
- 你的磁盘
- 你的 shell
- 你上一次的会话

文本输入，文本输出。框架处理其余部分。

layout: default

Aider 就是一个

Aider、模型和代码仓库



layout: default

回复

编辑形状的回复

```
path/to/file.py
<<<<<<< SEARCH
the lines exactly as they appear on disk
=======
the lines that replace them
>>>>>>> REPLACE
```

一个块，一个文件。没有任何东西强制集合必须是完整的。

layout: default

另一种形状

两种回复形状，每个模型设置一种 {.assert}

	整个	diff
模型返回	整个文件，每次	搜索和替换块
失败方式	删除你从未提到的行	与磁盘上的行不匹配
成本	每一行，每次编辑	只改变的行
设置为	4B	9B，如果你切换了它

一个框架设置，每个模型一行。这就是为什么会话不同。

layout: default

一般问题

通过文本通道的动作

模型返回	框架需要
散文	文件路径
散文	精确区域
散文	替换文本

搜索块、工具调用 JSON、标记输出。一个问题。
你的解析器是做什么将回复变成编辑，或拒绝它的。

layout: default

Aider 缺少什么

Aider 没有什么

Aider 没有	你在哪里见过它
周围的编辑器	Cursor
你输入时的补全	Cursor
超越会话的笔记	ChatGPT, Claude Code
自己寻找的文件	Cursor, Claude Code

每一行都是一个框架功能。它们都不是模型。

layout: section

上下文

谁决定模型看到什么

layout: default

两个选择者

谁选择每个部分

提示部分	选择者
系统提示	Aider，按模式
添加和只读文件	你，按命令
仓库地图	Aider，按排名
历史	你开始，Aider 修剪它
你的消息	你

你从未触碰的两行。其中一行 passed for intelligence。

layout: default

你的那一半

你控制什么

安排上下文	`/add` `/read-only` `/drop` `/clear` `/tokens`
更改你的文件	`/code` `/undo`

纯文本意味着 `/code`。默认写入。

大部分界面安排上下文。两个命令写入。

layout: default

当模型想要一个文件时

在 Aider 中，模型只能写入 {.assert}

	Aider	Claude Code
模型的动词	写入	读取、搜索、编辑、运行
想要一个文件	在散文中提到它的名称	调用读取工具
谁让它进入	Aider 发现名称，你说是	权限规则
什么进入提示	文件，每回合重新读取	文件，一次，当时的副本

Aider 做的其他一切都是围绕一个只能写入的模型添加的内容。

layout: default

Aider 的一半

它知道您从未添加过的文件 {.assert}

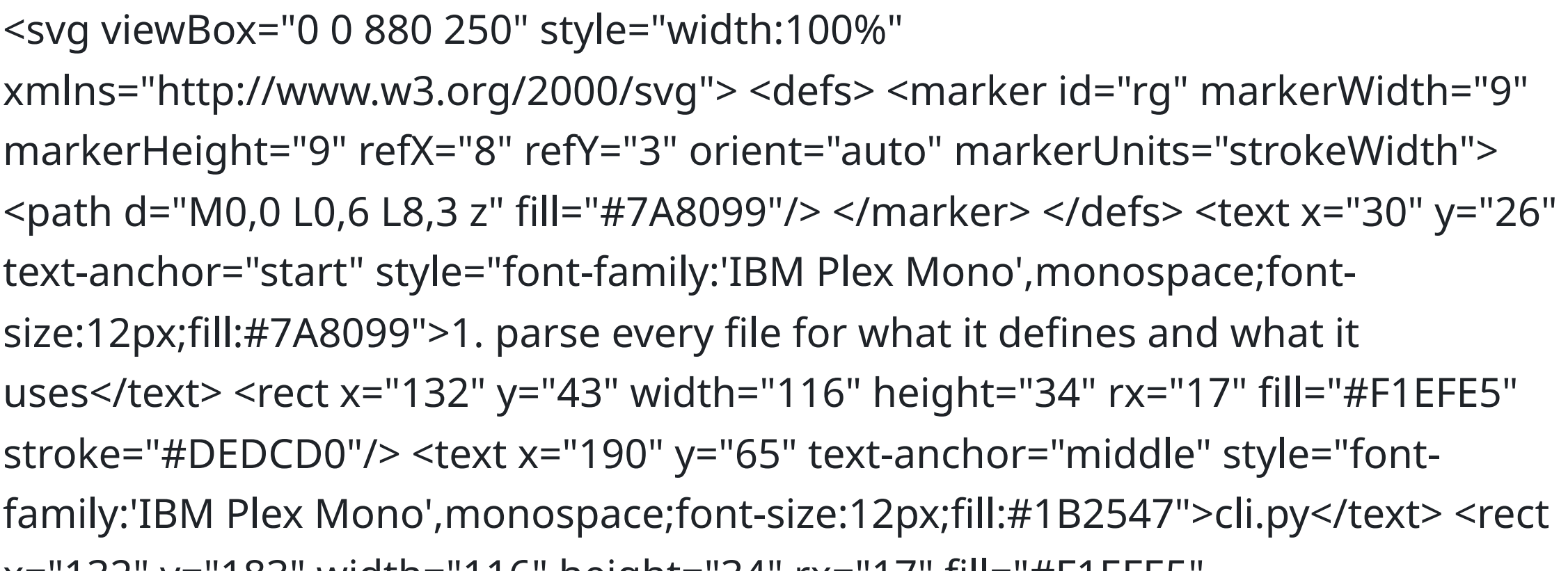
```
package/store.py:
⋮
│class Store:
│    def add(self, title, tags=()) -> Task:
│    def remove(self, task_id: int) -> bool:
⋮
│    def search(self, text, tag=None) -> list[Task]:
⋮

package/models.py:
│@dataclass
│class Task:
⋮
```

layout: default

排名

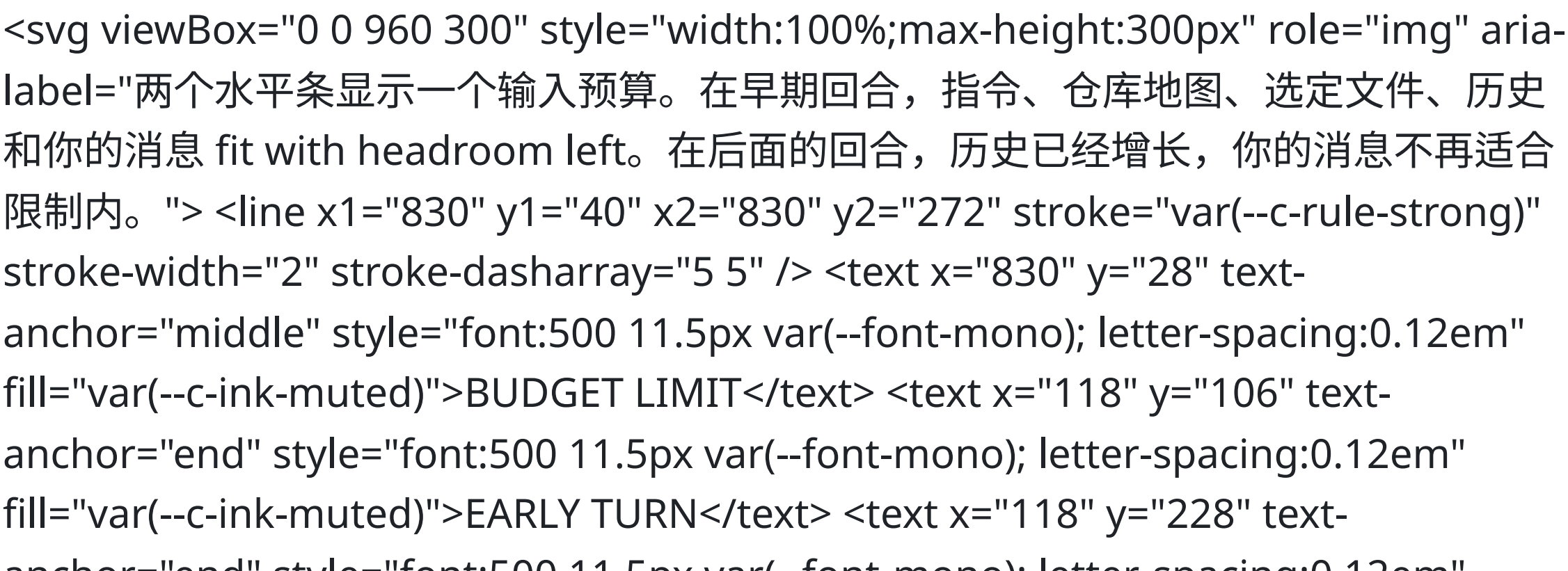
仓库地图的来源



layout: default

预算

一个共享的 token 预算



layout: section

维护提示

没有任何东西传递，所以每个部分每回合都重建

layout: default

按顺序

提示，从上到下



layout: default

每个回合

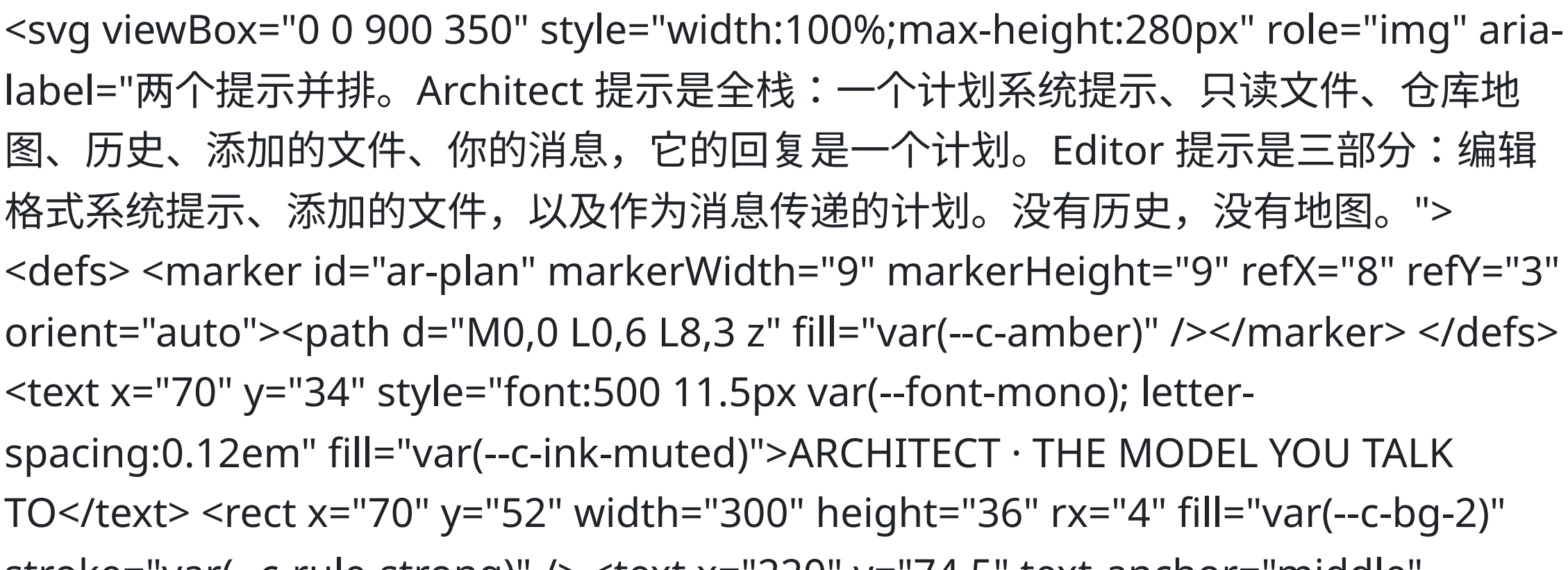
Aider 如何保持每个部分最新

部分	每个回合，Aider
系统提示	按模式交换：ask, code, architect
只读文件	从磁盘重新读取
仓库地图	围绕你刚刚命名的重新排名
历史	添加你的消息和它的回复，长时总结
添加的文件	从磁盘重新读取，整个
你的消息	附加格式提醒，窗口几乎满时丢弃

layout: default

两个提示

Architect 模式以干净的状态启动编辑器 {.assert}



layout: default

失败作为输入

为模型写的错误

Aider 回答模型，不是你。

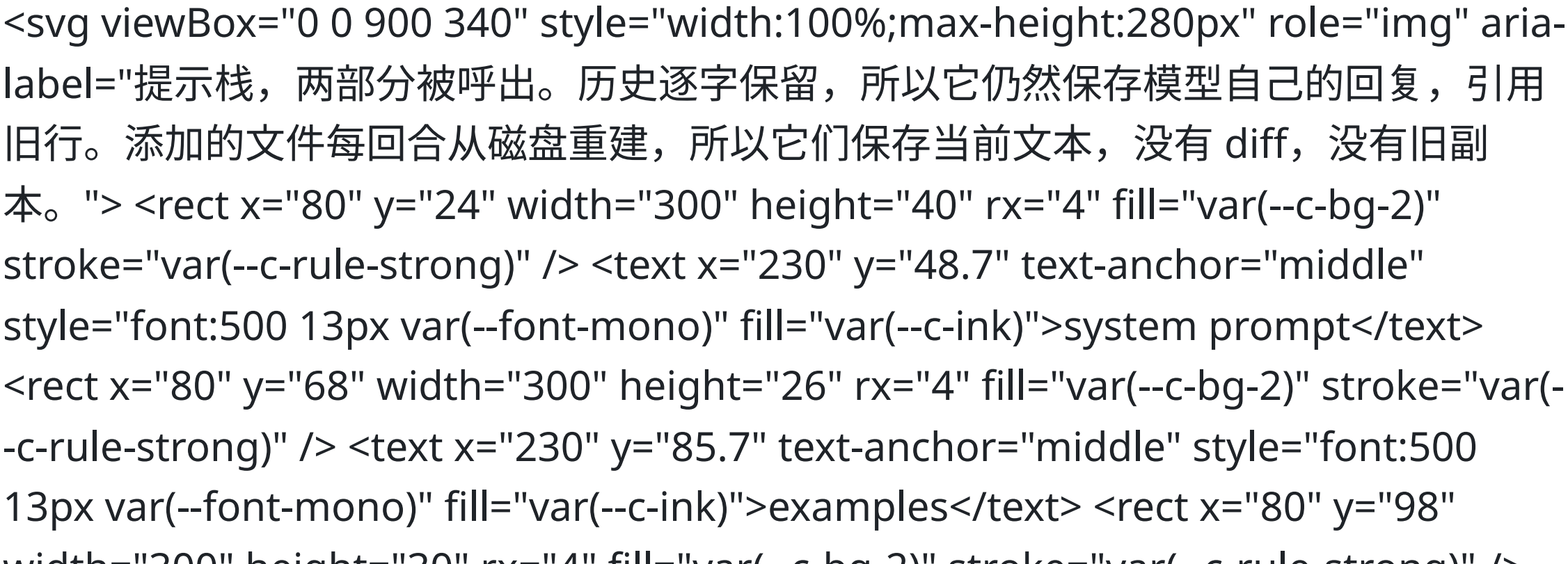
- 失败的块，引用回来
- 与文件几乎匹配的行
- 如果工作已经存在的说明

错误消息是一个提示。

layout: default

落地

编辑落地之后



layout: section

Hackathon 1

在你自己构建的一个功能，在一个晚上

layout: default

2026 年 9 月 24 日星期四

格式

- 两小时，你自己的项目
- 在房间里揭示的提示
- 离开前提交
- 与构建分开评分

layout: default

Hackathon 1

Hackathon 工作区

- 浏览器 VS Code，已经设置好
- Aider 指向课程端点
- 你的代码仓库已经克隆
- 一个 Python 参考包，离线

layout: section

结对程序员

Stage 1 要求什么

layout: default

Stage 1 · 项目

从零开始构建一个框架

你最后一小时的一切，由你编写。

它持有	你选择的一组文件
它询问	你配置的模型端点
它写入	只有在你批准之后

没有 Aider 的 Aider。同样的工作，你的代码。
截止日期：**2026 年 10 月 6 日星期二，晚上 11:59**。

layout: default

Stage 1 · 它如何运行

实现之前先写规范

你在写代码之前写需求，提交历史必须显示它。

你写的内容	何时	涵盖
系统设计	一次，实现之前	配置、运行契约、七个功能
三个图表	与设计一起	组件、序列、编辑生命周期
增量规范	在你选择的每个切片之前	有序、可测试、上下文范围

没有提供规范。

layout: default

评分

规范与设计

100 个构建点中的 50 个。

标准	分数
需求和接受标准	10
架构和界面设计	10
增量规范	10
三个图表	4 / 3 / 3
代码前的规范	5

layout: default

评分

实现与测试

100 个构建点中的 50 个。

标准	分数
功能正确性	25
自定义测试用例	9
失败路径测试	8
端到端运行和实时证据	5
操作说明	3

layout: default

你保留的内容

证据链

提交	代码之前的规范
会话	什么上下文，什么模型，发生了什么
调和	设计在哪里改变了，以及为什么

layout: default

模型规则

规范的模型规则

- 起草、 critique、图表、代码、测试
- 推荐 9B，允许 4B
- 次要模型也记录
- 允许任何兼容的主机

不同的提供者不是不同的模型。

layout: section

自主性，以及你的框架走向何方

框架变得更好。你将自己构建它。

layout: default

自主性

自主性是 handed to the model 的决策 {.assert}

框架决定	2023 年，Aider	2025 年，一个 agentic CLI
模型看到什么	你，用 `/add`	模型询问，并读取它
对回复做什么	一个编辑，先展示给你	一个工具调用，对照规则检查
接下来发生什么	你输入下一个请求	模型选择下一个动作
什么在会话中幸存	什么都没有	笔记、记忆、它可以恢复的运行

模型调用没有改变形状。它周围的一切都改变了。

layout: default

里程碑

项目里程碑

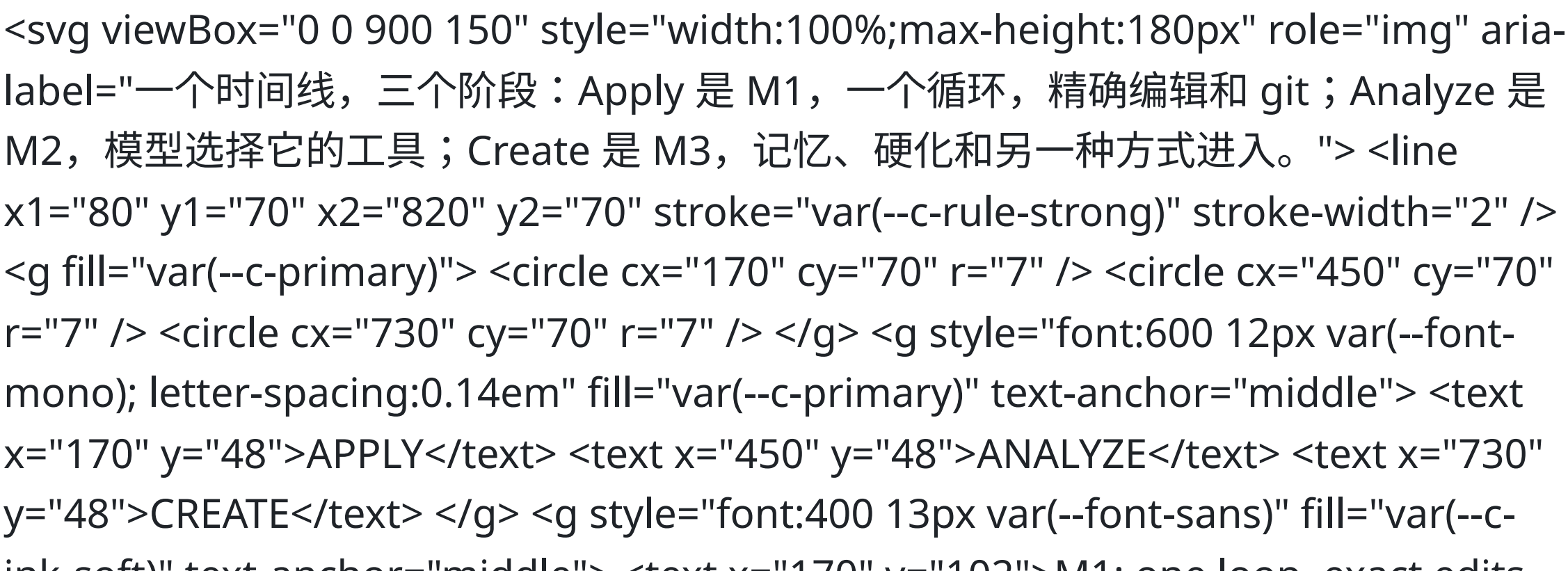
里程碑	谁采取步骤	当你构建它时
M1. 结对程序员	你选择文件并批准编辑；模型写入	Apply，从本周开始
M2. 智能体	模型选择下一个动作；你写的规则批准它	Analyze
M3. 助手	它在运行之间保持状态并幸存于真实工作	Create

更大的模型买不到这些。每个里程碑都是你写的框架。

layout: default

到十二月

一个代码仓库到十二月



layout: default

目的地

你将知道 Claude Code 如何工作 {.assert}

到十二月：一个 M3 框架，在 Claude Code 的形状中，从你这周开始的 M1 结对程序员成长。

使用它	知道它
批准一个权限提示	写它执行的规则
观察它找到文件	写找到它的搜索
打字，文件改变	写改变它的解析器

其他所有人都会使用它。你将编写每一层。

layout: default

你将遇到的每个工具

同样的循环，更多的机制

- 组装、回答、解析、应用、修复、提交
- 你将遇到的每个工具运行它
- 不同的是谁采取每个步骤

留给 L05：模型如何获得选择下一个动作，以及它花费什么。

layout: section

办公时间

会话的其余部分是你们的

layout: end